

Токенизация в языке C++

Автор: Рыжаков Никита

Этапы компиляции

Компиляция:

Input: `app.cpp`

Этапы компиляции

Компиляция:

Input: `app.cpp`

Output: `app.exe`

Этапы компиляции

Компиляция:

Input: `app.cpp`

Output: `app.exe`

Что под капотом?

Этапы компиляции

Компиляция:

Input: app.cpp

Preprocessing

Compilation

Assembly

Linking

Output: app

Этапы компиляции

Компиляция:

Input: app.cpp

Preprocessing

Compilation

Assembly

Linking

Output: app



Токенизация

iA+++iB

Токенизация

`iA+++iB`

что увидит компилятор?

Токенизация

iA+++iB

iA ++ + iB

Токенизация

`iA+++iB`

`iA ++ + iB`

токенизация реализована
жадным образом

Токенизация

после разбития
исходного кода на
токены, их границы
не меняются, но...

Препроцессорные директивы

```
#define foo() bar  
foo()baz // bar baz not barbaz
```

Препроцессорные директивы

```
#define foo() bar  
foo()baz // bar baz not barbaz
```

токен после препроцессинга не
ретокенизируется. Компилятор
воспринимает результат как
единый токен

Препроцессинговые токены

- идентификаторы
- числа препроцессинга
- строковые литералы
- знаки пунктуации
- прочее

Препроцессинговые токены

- идентификаторы
- числа препроцессинга
- строковые литералы
- знаки пунктуации
- прочее

Ключевые слова языка C
воспринимаются как идентификаторы
(кроме `#define`)

Идентификаторы

Table 4 — Keywords

alignas	continue	friend	register	true
alignof	decltype	goto	reinterpret_cast	try
asm	default	if	return	typedef
auto	delete	inline	short	typeid
bool	double	int	signed	typename
break	do	long	sizeof	union
case	dynamic_cast	mutable	static	unsigned
catch	else	namespace	static_assert	using
char	enum	new	static_cast	virtual
char16_t	explicit	noexcept	struct	void
char32_t	export	nullptr	switch	volatile
class	extern	operator	template	wchar_t
const	false	private	this	while
constexpr	float	protected	thread_local	
const_cast	for	public	throw	

Идентификаторы

через `#define` мы можем определять макросы, чьи названия совпадают с ключевыми словами языка C.

Препроцессинговые ТОКЕНЫ

через `#define` мы можем определять макросы, чьи названия совпадают с ключевыми словами языка C. Кроме

- `and`
- `and_eq`
- `bitand`
- `bitor`
- `compl`
- `not`
- `not_eq`
- `or`
- `or_eq`
- `xor`
- `xor_eq`

Идентификаторы

- можно использовать юникод (начиная со стандарта C99)
- “\$” в GCC воспринимается как строковый литерал (в стандарте не прописано, как компилятору себя вести)

Препроцессинговые числа

- всевозможные числовые константы из языка C

Строковые литералы

- СИМВОЛЬНЫЕ КОНСТАНТЫ
- строковые константы
- имена header файлов

Знаки пунктуации

- привычные нам знаки пунктуации
- биграммы:
 - <% : {
 - %> : }
 - <: : [
 - :> :]
 - %: : #
 - %:%: : ##

Забавные примеры

```
#define R "x"  
const char* s = R"y"; // ill-formed raw string, not "x" "y"
```

1Ex // a preprocessing number token

```
0xe+foo // 0xe + foo
```

```
std::vector<std::vector<int>> // раньше нужен был пробел после ...<int>
```

Комментарии

- `/* */` – многострочные
- `//` – однострочные
- и как бы никакой вложенности нет

Заглянем в CLang

- `class Token` (реализован согласно стандарту; пруфы тут: [\[1\]](#), [\[2\]](#)) в `clang::Lexer`

Заглянем в CLang

- `class Token` (реализован согласно стандарту; пруфы тут: [\[1\]](#), [\[2\]](#)) в `clang::Lexer`
- есть поддержка чтения текста файла без препроцессинга или парсинга macro определений

Заглянем в CLang

```
#include "clang/Basic/LangOptions.h"
#include "clang/Lex/Lexer.h"
#include "clang/Lex/Token.h"
#include <string>
#include <iostream>

void lexWithClang(const std::string& code) {
    clang::LangOptions langOpts;
    const char* start = code.data();
    const char* end = code.data() + code.size();

    clang::SourceLocation loc;
    clang::Lexer lexer(loc, langOpts, start, start, end);

    clang::Token tok;
    while (!lexer.LexFromRawLexer(tok)) {
        if (tok.is(clang::tok::eof)) break;

        std::cout << "Token Category: " << tok.getName()
                  << " | Length: " << tok.getLength() << "\n";
    }
}
```

Кратко про Go1ang

- жадным образом работает с SSA
- контекстно независимо
- подробнее можно глянуть в `cmd/compile/internal/syntax` и в `cmd/token` (не путать с `go/scanner` – это для внешних тулзов)

Кратко про Go1ang

```
type Token struct {  
    kind TokenKind // _Func, _Ident, _Plus, etc.  
    pos  uint32       // encoded file position  
    lit  string       // raw text of the token (for identifiers, keywords, literals)  
}
```


Кратко про Go1ang

- можно не ставить точки с запятыми :)

```
x := 5
```

```
y := x + 1
```

Кратко про Go1ang

- МОЖНО НЕ СТАВИТЬ ТОЧКИ С ЗАПЯТЫМИ (почти)

```
x := func() {  
    return 5  
}()
```

Пример (с тащил с гитхаба golang)

```
package token_test

import (
    "fmt"
    "go/ast"
    "go/parser"
    "go/token"
)

func Example_retrievePositionInfo() {
    fset := token.NewFileSet()

    const src = `package main

import "fmt"

import "go/token"

//line :1:5
type p = token.Pos

const bad = token.NoPos

//line fake.go:42:11
func ok(pos p) bool {
    return pos != bad
}

/*line :7:9*/func main() {
    fmt.Println(ok(bad) == bad.IsValid())
}
`
```

```
f, err := parser.ParseFile(fset, "main.go", src, parser.SkipObjectResolution)
if err != nil {
    fmt.Println(err)
    return
}

// Print the location and kind of each declaration in f.
for _, decl := range f.Decls {
    // Get the filename, line, and column back via the file set.
    // We get both the relative and absolute position.
    // The relative position is relative to the last line directive.
    // The absolute position is the exact position in the source.
    pos := decl.Pos()
    relPosition := fset.Position(pos)
    absPosition := fset.PositionFor(pos, false)

    // Either a FuncDecl or GenDecl, since we exit on error.
    kind := "func"
    if gen, ok := decl.(*ast.GenDecl); ok {
        kind = gen.Tok.String()
    }

    // If the relative and absolute positions differ, show both.
    fmtPosition := relPosition.String()
    if relPosition != absPosition {
        fmtPosition += "[" + absPosition.String() + "]"
    }

    fmt.Printf("%s: %s\n", fmtPosition, kind)
}

// Output:
//
// main.go:3:1: import
// main.go:5:1: import
// main.go:1:5[main.go:8:1]: type
// main.go:3:1[main.go:10:1]: const
// fake.go:42:11[main.go:13:1]: func
// fake.go:7:9[main.go:17:14]: func
}
```


Кратко про Rust

- жадник, чекай `rustc_lexer`

Кратко про Rust

- жадник, чекай `rustc_lexer`
- из забавного: никогда не кинет ошибку :)

Кратко про Rust

- жадник, чекай `rustc_lexer`
- из забавного: никогда не кинет ошибку :)
- препроцессинг есть, но отправливает AST токены

Кратко про Rust

- жадник, чекай `rustc_lexer`
- из забавного: никогда не кинет ошибку :)
- препроцессинг есть, но отправливает AST токены
- контекстно независимо

Кратко про Rust

- жадник, чекай `rustc_lexer`
- из забавного: никогда не кинет ошибку :)
- препроцессинг есть, но отправливает AST токены
- контекстно независимо

нет, кода на слайде не будет

Спасибо за внимание!!!